SYSTEMS INTEGRATION · LAB 0 · SETUP GUIDE

Lab 0 Guide
Mini-ERP: local and on SAP BTP

What did we do, why, and in which order? SAP HANA, Cloud Foundry, Business Application Studio and the connections between them – step by step, for setting everything up again.

Sources: `lab/mini-erp.md` in the lecture repository, `src/simple-erp/mta.yaml`, `package.json`, `xs-security*.json` and the real setup in our BTP trial account.

Account data (our setup): global account / org `369eca91trial`, subaccount `trial`, Cloud Foundry space dev, region us10.

Language: English; SAP terms are used as they appear on the screen.

Contents

1. Summary on one page . . . 3

2. The big picture: two environments, one ERP . . . 4

What is inside the Mini-ERP? . . . 4

3. What is the local part (cds watch) for? . . . 5

Commands (on your computer, once) . . . 5

4. BTP concepts: the confusing part . . . 6

5. Why did we set up SAP HANA? Wasn't BTP "dev" enough? . . . 7

5.1 Why is Cloud Foundry alone not enough? . . . 7

5.2 Why is BAS (the dev space) not enough? . . . 7

6. How were HANA and Cloud Foundry connected? . . . 8

6.1 The connection chain, step by step . . . 8

6.2 Did we change any setting in HANA? . . . 9

7. Which commands did we run in BAS, and why? . . . 10

7.1 Preparation (once) . . . 10

7.2 Terminal commands . . . 10

7.3 Assigning the roles (in the cockpit) . . . 11

8. How did the web shop get access to the ERP API? (link to Lab 1) . . . 11

9. Start order every day / before a presentation . . . 12

10. Setup from scratch – checklist . . . 12

11. Common errors and solutions . . . 13

12. Command card and glossary . . . 14

1. Summary on one page

In Lab 0 we ran the small ERP system prepared for the lecture (**Mini-ERP**) in two places: **on our own computer** (for development and testing) and **in the SAP BTP cloud** (a real, persistent installation that can be reached from the internet). The Mini-ERP manages products, stock, customers and orders. The web shop of Lab 1 connects to this ERP: it reads the products from it and writes the orders into it.

Question	Short answer
What is the local part for?	Development and testing. `cds watch` starts it in seconds; the database lives in memory (SQLite in-memory), the users are mocked (alice, service-user). Stopping it resets the data.
Does the web shop read its data from there?	Yes. While the connector “Local SAP (cds watch)” is active in the local shop, the products come from `localhost:4004`, and buying a bike **reduces the stock there**. The cloud shop uses the ERP on BTP (HANA).
Why SAP HANA?	In the cloud the disk of an application is temporary; data must live in a persistent database. The ERP’s “production” profile requires HANA. SQLite is only for local development.
Wasn’t the BTP “dev” part enough?	No. There are two different “dev” things: **BAS dev space** = VS Code in the cloud (where code is written and built); **Cloud Foundry space dev** = where the applications run. Neither stores data; the data lives in HANA.
How did we connect HANA?	We did not write a connection by hand. We created the HANA instance mapped to our CF space; `cf deploy` opened an **HDI container** in HANA and connected it to the ERP application with a **service binding**. The credentials arrived automatically as an environment variable (VCAP_SERVICES).
What do I have to do every day?	In the trial account everything stops every night. In this order: start HANA → `cf start simple-erp-srv` (→ for the shop `cf start velocity-webshop`).

> **How to read this guide**
> Sections 2–4 answer “what is what?”, sections 5–7 answer “how is it connected and which commands?”. Section 10 is the checklist for a setup from scratch; section 11 lists common errors, section 12 is the command card.

2. The big picture: two environments, one ERP

The same Mini-ERP code (SAP CAP, Node.js) runs with two different “profiles”. The setting `cds.requires` in `package.json` decides which profile is used:

```
"cds": {
  "requires": {
    "[development]": {
      "auth": { "kind": "mocked", "users": { "alice": ..., "service-user": ... } }
    },
    "db": { "kind": "sqlite", "credentials": { "url": ":memory:" } },
    "[production]":  { "db": "hana", "auth": "xsuaa" }
  }
}
```

src/simple-erp/package.json (shortened): development = SQLite + mocked users, production = HANA + XSUAA

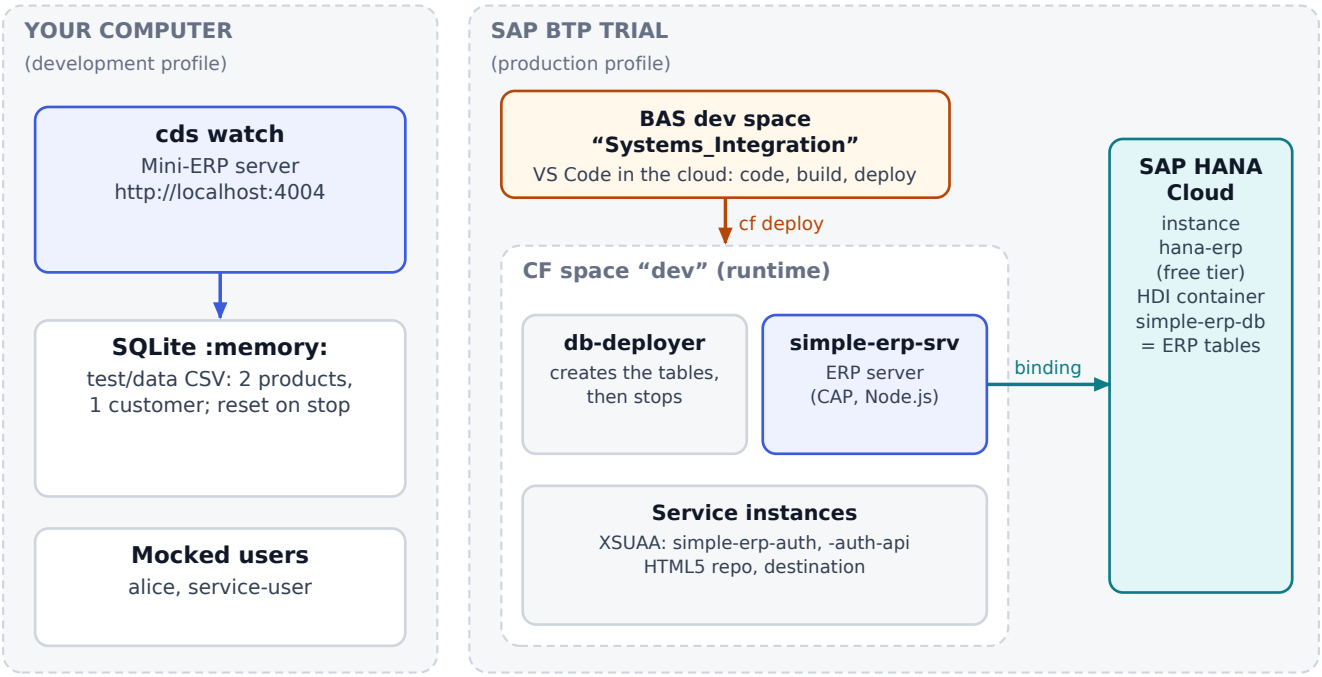


Figure 1 – The same Mini-ERP in two places: the local development environment on the left, the installation on SAP BTP on the right.

What is inside the Mini-ERP?

- **Data model** (`db/simpleerp-schema.cds`): Products (productID, name, description, price, currency, stock 0–1000), Customers (name, email, address), Orders (orderID, customer, orderDate, orderAmount, orderStatus: NEW → PICKED → SHIPPED → COMPLETED / CANCELED) and OrderItems.
- **Services** (`srv/`): the OData v4 service `/odata/v4/simple-erp` (read/write, order actions pickOrder, shipOrder, completeOrder, cancelOrder), the REST API `/rest/api` (e.g. `products()`) and a UI service for the Fiori apps.
- **Fiori apps** (`app/products, app/orders, app/customers`): screens for products, orders and customers.
- **Authorizations**: role `customerservice` (Fiori screens) and `system-integration` (API access).

3. What is the local part (cds watch) for?

The local installation is for **trying out, developing and testing the ERP quickly**. It needs no cloud account, no internet and no HANA; it starts in seconds. While developing Lab 1 we connected the shop to it first.

Property	Local (development)	Why it matters
Start	`cds watch`	Restarts itself when a file changes (ideal for development).
Database	SQLite, `:memory:`	Starts empty every time and loads `test/data/*.csv`: 2 products (MTB, Gravel) and 1 customer (Christian Drumm).
Users	mocked: alice/alice, bob/bob, service-user/service-user	No real identity service (XSUAA); the browser asks with basic auth.
Address	http://localhost:4004	Reachable from your computer only.
Persistence	None	After a restart, stock and orders are back to the initial data of the lecture.

Commands (on your computer, once)

```
npm install -g @sap/cds-dk                      # SAP CAP command line tool (cds)
git clone https://codeberg.org/ceedee666/systems-integration.git
cd systems-integration/src/simple-erp
npm install                                      # dependencies of the project
cds watch                                        # start the ERP -> http://localhost:4004
```

On our computer the repository is already in C:\Users\ONUR\systems-integration; each time only cd ... and npx.cmd cds watch are needed (.cmd because PowerShell blocks the npm.ps1/npx.ps1 scripts).

> **Relation to the web shop**
>
> The local shop (`localhost:5173`) uses the connector **Local SAP (cds watch)** by default: it reads the products from here with `GET /rest/api/products()` and, when an order is placed, writes the order here with `POST /odata/v4/simple-erp/Orders`. The **ERP** checks and reduces the stock; the shop then reads the new stock again.
>
> That is why the stock goes down in the local ERP when you buy a bike – but it is reset when `cds watch` restarts. When the ERP restarts, the shop loads its demo catalog (14 products) into the ERP again automatically; the original lecture data are only MTB and Gravel.

Practical shortcut: double-clicking `start-local.cmd` in the `webshop` folder starts both the ERP (`cds watch`) and the shop.

4. BTP concepts: the confusing part

SAP BTP has many layers with similar names. The biggest source of confusion: **“dev space” and “space dev” are different things**. Use the table below as a map.

Concept	Our name	What is it?	Analogy
Global account	369eca91trial	The top-level account at SAP (your trial account).	Company
Subaccount	trial (us10)	The sub-account where services are enabled; the region is chosen here.	Building
Cloud Foundry org	369eca91trial	The Cloud Foundry environment inside the subaccount.	Floor
CF space	**dev**	Where the applications **run** and the service instances are created. `cf apps` lists this space.	Room (the machines run here)
BAS dev space	**Systems_Integration**	SAP Business Application Studio: VS Code in the browser. Code, terminal, `mbt` and `cf` are ready to use. Applications do **not run** here, they are **deployed** from here.	Workshop / workbench
SAP HANA Cloud	hana-erp	Managed database service (free tier). The data is stored here persistently.	Warehouse
HDI container	simple-erp-db	Isolated schema (tables) of the ERP inside HANA. Created during the deployment.	Your own shelf in the warehouse
XSUAA	simple-erp-auth, simple-erp-auth-api	Identity and authorization service (OAuth 2.0): roles and tokens.	Security guard
HTML5 repo + destination	simple-erp-html5-repo-host, simple-erp-destination-service	Hosts the Fiori screens and routes them to the server.	Shop window
SAP Build Work Zone	(subscription)	Launchpad for Fiori apps. Our subscription is “failed”; it is not needed for the API or the web shop.	Main entrance

Watch out for the word “dev”

BAS dev space (Systems_Integration): like your development computer; it sleeps when not used, nobody on the internet can connect to it, and its data is not ERP data.

CF space dev: the server environment where our applications (`simple-erp-srv`, `velocity-webshop`) run 24/7. In the cockpit: Subaccount → Cloud Foundry → Spaces → dev.

5. Why did we set up SAP HANA? Wasn't BTP "dev" enough?

Short answer: **running an application and storing data are different jobs.** Cloud Foundry runs the application, HANA stores the data. BAS is only a development tool.

5.1 Why is Cloud Foundry alone not enough?

- **The disk is temporary:** in CF every application runs in a container. After `cf restart`, `cf push` or the nightly stop, the container's disk is reset. ERP data stored there would be lost every time. (This is exactly why the SQLite database of our cloud web shop is reset on every restart – the ERP data is in HANA, so products and orders are not lost.)
- **Several instances:** if an application runs in 2–3 copies, all of them must see the *same* data. Only a shared database makes that possible.
- **The ERP is configured that way:** the `[production]` profile says `"db": "hana"`. In the cloud the CAP server looks for a HANA binding and does not start without it.
- **Enterprise features:** HANA provides backups, transaction safety, concurrent access and SAP's standard data layer (HDI).

5.2 Why is BAS (the dev space) not enough?

- BAS is a **development environment**: you write code, package it with `mbt build` and upload it with `cf deploy`. It stops itself when it is not used.
- Running `cds watch` in BAS is possible too, but that is "local" mode again (SQLite, mocked users) and only a preview for you. It is not a system that the lecturer, team mates or the web shop can use 24/7.

	Local (cds watch)	BAS dev space	CF space dev + HANA
Purpose	Fast development/testing	Development and deployment tool	Real operation (production)
Where is the data?	In memory (SQLite)	— (code only)	HANA (persistent)
Users	Mocked	—	Real: XSUAA + role collection
Who has access?	Only you	Only you	Over the internet (every authorized user, the web shop)
When stopped	Data is reset	Code stays	Data stays (the trial stops at night, data is not deleted)

6. How were HANA and Cloud Foundry connected?

The key point: **we did not write the connection by hand.** There is no HANA password in the code or in any configuration file. Three pieces created the connection: the mapping of the HANA instance to the space, `mta.yaml` and `cf deploy`.

6.1 The connection chain, step by step

1. **We created the HANA instance mapped to the Cloud Foundry space.** The “Important Note” in the lab text: when the instance is created, *Runtime Environment = Cloud Foundry* and the right org/space (ours: dev) must be selected. Then the “hana” service (plan `hdi-shared`) can be used in that space. In HANA Cloud Central our instance appears as “hana-erp, Runtime Environment: Cloud Foundry”.
2. `mta.yaml` **requests an HDI container:** resource `simple-erp-db` → type `com.sap.xs.hdi-container`, service hana, plan `hdi-shared`.
3. `cf deploy` **creates this container inside HANA** (an isolated schema of the ERP in HANA).
4. `simple-erp-db-deployer` (module type hdb) connects to the container, creates the tables and views and **stops** when it is done. That is why “stopped” in the cockpit is normal.
5. The `requires` list of the `simple-erp-srv` module contains `simple-erp-db` and `simple-erp-auth` → CF **binds** them to the application (service binding).
6. Because of the binding, Cloud Foundry puts the connection data (host, port, user, password, schema, certificate) into the VCAP_SERVICES environment variable of the application. CAP (`@cap-js/hana`) reads it **automatically** and connects to HANA.

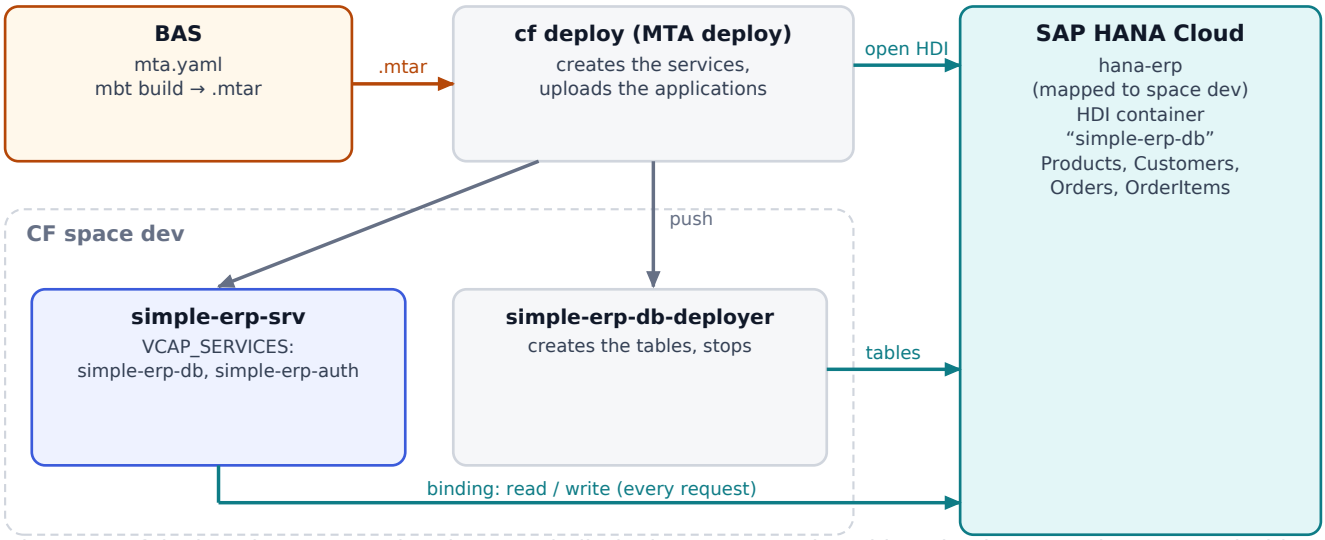


Figure 2 – cf deploy: the HDI container is opened, db-deployer creates the tables, simple-erp-srv is connected with a binding.

6.2 Did we change any setting in HANA?

Setting	What we did / what it must be	Why?
Runtime Environment / mapping	**Cloud Foundry**, org 369eca91trial, space **dev**	Without it `cf deploy` cannot create the HDI container and the deployment fails.
Plan	Free tier (hana-free)	Free for the trial; stops automatically every night.
Allowed connections (IP)	“Allow only BTP IP addresses” is enough for CF applications.	The ERP server (simple-erp-srv) connects from inside BTP. To connect to HANA directly from your own computer (DBeaver, hybrid testing) you would need “Allow all IP addresses” or your own IP. The web shop never connects to HANA directly; it only calls the ERP’s API, so the shop needs no IP setting.
User/password	None (nothing entered by hand)	The HDI container creates its own technical users; they arrive with the binding.
Daily task	HANA Cloud Central → hana-erp → ⋯ → **Start**	The trial stops at night; HANA must be “Running” before a deployment or an ERP request.

7. Which commands did we run in BAS, and why?

Order: open the dev space → clone the repository → install the dependencies → build the package → log in to Cloud Foundry → deploy → assign the roles. The commands are typed in the BAS terminal (Terminal → New Terminal).

7.1 Preparation (once)

1. Create an SAP BTP trial account (tutorial linked in the lab).
2. Subscribe to Business Application Studio and create a dev space of type **Full Stack Cloud Application** named `Systems_Integration`.
3. Add the HANA Cloud subscription (tools plan) and create an instance in HANA Cloud Central – **mapped to the Cloud Foundry space dev** (section 6.2).
4. (Optional) SAP Build Work Zone subscription – for the Fiori launchpad. Not needed for the API.

7.2 Terminal commands

```
# 1) Clone the repository (or Source Control > Clone Repository) into /home/user/projects
cd ~/projects
git clone https://codeberg.org/ceedee666/systems-integration.git

# 2) Install the npm packages of the ERP and of the 3 Fiori apps
cd systems-integration/src/simple-erp
npm install
(cd app/customers && npm install)
(cd app/orders && npm install)
(cd app/products && npm install)

# 3) Build: cds build --production + Fiori build -> mta_archives/*.mtar
mbt build

# 4) Log in to Cloud Foundry (API endpoint: on the subaccount's Overview page)
cf login -a https://api.cf.us10-001.hana.ondemand.com      # for SSO: --sso
#    choose org 369eca91trial, space dev

# 5) Deploy (HANA must be "Running"!)
cf deploy mta_archives/simple-erp_1.0.0.mtar

# 6) Check
cf apps          # simple-erp-srv: started, simple-erp-db-deployer: stopped (normal)
cf services      # simple-erp-db, simple-erp-auth, simple-erp-auth-api, ...
```

Alternative: the package.json scripts – npm run build (→ mta_archives/archive.mtar) and npm run deploy.

Command	What it does	Why it is needed
`npm install`	Installs the node_modules folder.	The build does not work without CAP and its tools.
`mbt build`	Reads `mta.yaml`; creates `gen/srv` and `gen/db` with `npx cds build --production`, builds the Fiori apps and puts everything into one `.mtar` archive.	Cloud Foundry expects one package (MTA).
`cf login`	Connects to CF and selects org/space.	Decides into which account and space the deployment goes.

Command	What it does	Why it is needed
`cf deploy`	Creates the services (HDI, 2× XSUAA, HTML5 repo, destination), uploads the applications, creates the bindings, runs the db-deployer and puts the Fiori apps into the HTML5 repo.	The whole installation in one command, repeatable, as described in `mta.yaml`.

7.3 Assigning the roles (in the cockpit)

1. Subaccount → Security → **Role Collections** → **Create**: name `Simple ERP All`.
2. Add the roles `customerservice` and `system-integration` (search with the application identifier `simple-erp`).
3. Assign the role collection to your user. Then the Fiori apps open; otherwise you get 403 Forbidden.

8. How did the web shop get access to the ERP API? (link to Lab 1)

The web shop is not a person but an **application**. That is why it uses the **OAuth 2.0 client credentials** flow instead of a user name and password: it gets an application token from XSUAA and adds it to every ERP request as `Authorization: Bearer <token>`.

- `mta.yaml` creates a second XSUAA service: `simple-erp-auth-api` (`xs-security-api.json`: xsappname `xsappforapi`, authorities `$ACCEPT_GRANTED_AUTHORITIES`).
- The ERP's own security file (`xs-security.json`) grants the authority `system-integration` to this application with `grant-as-authority-to-apps: $XSAPPNAME(application, xsappforapi)`.
- Result: a token obtained with the credentials of `simple-erp-auth-api` contains the authority `system-integration` → the REST/OData API accepts it.

```
# Shop on the computer, ERP in the cloud: service key = JSON file
cf create-service-key simple-erp-auth-api webshop-key
cf service-key simple-erp-auth-api webshop-key
#   -> save the JSON of the output as server/xsuaa-key.json (NEVER share or commit it)

# Shop in the cloud: no key file, a binding (manifest.yml -> services: simple-erp-auth-api)
cf push            # in the cf-dist folder; the credentials arrive with VCAP_SERVICES
```

9. Start order every day / before a presentation

> **In the trial account everything stops every night**
> The HANA instance and the applications are stopped automatically at night. The order matters: first the database, then the applications that connect to it.

1. **HANA:** BTP cockpit → Instances and Subscriptions → SAP HANA Cloud → *Go to Application* (HANA Cloud Central) → hana-erp → ⋯ → **Start**. Wait until it is "Running" (5–10 min).
2. **ERP server:** in PowerShell (with cf installed) or in the BAS terminal: `cf start simple-erp-srv`
3. **Cloud shop:** `cf start velocity-webshop`
4. **Check:** `cf apps` → both *started*.
5. **Local side:** double-click `webshop\start-local.cmd` (local ERP + local shop).

```
cf login -a https://api.cf.us10-001.hana.ondemand.com --sso   # if needed
cf start simple-erp-srv
cf start velocity-webshop
cf apps
```

10. Setup from scratch – checklist

#	Step	Where?	Check
1	BTP trial account + subaccount (us10)	SAP BTP cockpit	The CF API endpoint is shown on the subaccount Overview
2	Cloud Foundry environment + space **dev**	Subaccount → Cloud Foundry	dev is listed under Spaces
3	Business Application Studio subscription + dev space "Systems_Integration" (Full Stack)	Instances and Subscriptions	RUNNING in the Dev Space Manager
4	SAP HANA Cloud subscription (tools)	Service Marketplace	HANA Cloud Central opens
5	HANA instance (free tier) – **Runtime: Cloud Foundry, space dev**	HANA Cloud Central	State: Running
6	Clone the repository in BAS, 4× `npm install`	BAS terminal	node_modules exists
7	`mbt build`	BAS terminal	mta_archives/*.mtar was created
8	`cf login` (org 369eca91trial, space dev)	BAS terminal	`cf target`
9	`cf deploy mta_archives/...mtar`	BAS terminal	"Process finished"; cf apps
10	Role collection "Simple ERP All" (customerservice + system-integration) → your user	Security → Role Collections	The Fiori apps do not return 403
11	(Lab 1) API service key, or `cf push` of the shop	PowerShell/BAS	"Live stock from SAP BTP" in the shop

11. Common errors and solutions

Symptom	Cause	Solution
`cf deploy` fails with an HDI/hana error	HANA is stopped or the instance is not mapped to the space	Start HANA; check the instance mapping (CF, space dev)
Browser: “404 Not Found: Requested route ... does not exist”	The application is stopped	`cf start simple-erp-srv`
Fiori apps return 403 Forbidden	The role collection is not assigned	Assign Simple ERP All to your user, log in again
API 401 Unauthorized	Wrong or missing token / service key	Get the service key again; Test the connector in the shop
`cds` / `cf` “not recognized”	The tool is not installed or not on the PATH	`npm i -g @sap/cds-dk`; for cf the PATH line (Lab 1 guide)
PowerShell: “running scripts is disabled” (npm.ps1)	Windows script execution policy	Use `npm.cmd` and `npx.cmd`
Stock/orders disappeared in the local ERP	cds watch restarted (in-memory)	Normal; the shop loads the demo catalog again
Work Zone subscription “failed”	Needs an IAS trust / extra setup	Not needed for the Lab 0 API or the web shop
`simple-erp-db-deployer` “stopped”	Its job is done	Normal, leave it

12. Command card and glossary

```
# --- Local ---
cd C:\Users\ONUR\systems-integration\src\simple-erp
npx.cmd cds watch          # http://localhost:4004 (alice/alice, service-user/service-user)

# --- Cloud (BAS terminal or PowerShell with cf installed) ---
cf login -a https://api.cf.us10-001.hana.ondemand.com --sso
cf target                              # which org/space?
cf apps                                # applications and their state
cf services                            # service instances
cf start simple-erp-srv                # start the ERP (HANA must be Running)
cf logs simple-erp-srv --recent        # recent logs (when looking for errors)
mbt build && cf deploy mta_archives/simple-erp_1.0.0.mtar   # deploy again
```

Term	Meaning
SAP CAP	Cloud Application Programming Model: SAP’s Node.js/Java framework. Model, services and APIs with CDS.
CDS	Core Data Services: the language in which the data model and the services are defined (.cds files).
OData v4	REST-based standard data protocol used by SAP ($filter, $select, actions).
MTA / .mtar	Multi-Target Application: one package for an application made of several modules + services.
mbt	MTA Build Tool: creates the .mtar from mta.yaml.
HDI container	HANA Deployment Infrastructure: an isolated database schema that belongs to one application.
Service binding	Connecting a service to an application; the credentials arrive with VCAP_SERVICES.
VCAP_SERVICES	JSON environment variable in which Cloud Foundry passes the data of the bound services to the application.
XSUAA	The OAuth 2.0 authorization service of BTP: roles, scopes, tokens.
Service key	JSON credentials of a service for use from outside (e.g. the shop on your computer).
Role collection	Groups roles so they can be assigned to users (Simple ERP All).